

# The Edge LLM Pattern

## Autonomous Agents as First-Class MCP Servers

*Architecture Reference | edge-llm-core v0.1.0*

---

### 1. Abstract

The Model Context Protocol (MCP) standardises how host LLMs call external tools. In practice, every published MCP server today follows the same pattern: a stateless request arrives, data is fetched or written, and a response is returned. The host model does all the reasoning.

The Edge LLM Pattern breaks this assumption. Instead of a stateless adapter, an MCP server hosts its own internal LLM loop, domain state machine, and persistent memory. The host delegates an entire domain of responsibility to this specialist agent rather than making individual tool calls. The result is a fundamentally different unit of composition for agentic systems.

### 2. The Problem with Stateless MCP Servers

#### 2.1 The current model

Today's MCP ecosystem is built around stateless wrappers. A server exposes functions -search the web, query a database, send an email -and the host model decides when and how to call them. This model has three structural limits:

- Context saturation: Every tool result returns to the host's context window. Complex multi-step workflows consume tokens rapidly and degrade quality.
- Shallow delegation: The host must understand the domain deeply enough to orchestrate each step. It cannot truly delegate -it can only outsource individual actions.
- No memory: Each call is independent. There is no notion of a relationship building over time, a ticket progressing through a lifecycle, or a conversation accumulating context.

#### 2.2 What is missing

Enterprise workflows are not collections of atomic actions -they are processes with state, memory, and domain-specific reasoning accumulated over time. A sales pipeline, a support ticket, or an invoice approval flow involves dozens of interactions, each dependent on all prior context. No stateless MCP server can represent this naturally.

## 3. The Edge LLM Pattern

### 3.1 Core idea

An Edge LLM agent is an MCP server that contains its own autonomous LLM loop. When the host calls `handle_message`, it does not receive raw data -it receives the final output of a complete reasoning cycle that may involve multiple internal tool calls, state transitions, and memory updates.

The host model delegates domain ownership, not individual actions.

### 3.2 AS-IS vs TO-BE

| Dimension     | AS-IS (Stateless Wrapper)        | TO-BE (Edge LLM Agent)              |
|---------------|----------------------------------|-------------------------------------|
| Unit of call  | Single action                    | Entire domain interaction           |
| State         | None                             | Persistent per-entity state machine |
| Memory        | None                             | Per-entity conversation + metadata  |
| Reasoning     | Host LLM only                    | Specialist LLM inside the server    |
| Context cost  | High (all steps visible to host) | Low (only final reply surfaces)     |
| Multi-tenancy | Application concern              | Built-in with per-tenant config     |
| Autonomy      | None                             | Can act without further host input  |

### 3.3 Anatomy of an Edge LLM agent

Every agent built on this pattern is composed of five layers:

| Layer         | Responsibility                      | Default implementation         |
|---------------|-------------------------------------|--------------------------------|
| LLM Loop      | Reason, use tools, produce reply    | Claude Opus (configurable)     |
| State Machine | Track domain lifecycle stages       | StateMachine ABC               |
| Memory        | Persist conversation + entity state | JSON (dev) / Redis (prod)      |
| Tenant Store  | Per-tenant config and API keys      | JSON (dev) / PostgreSQL (prod) |
| Usage Tracker | Billing-ready token metering        | JSON (dev) / PostgreSQL (prod) |

### 3.4 Compaction

Long conversations are automatically compacted using a lighter model (Haiku by default) when the message count exceeds a configurable threshold. A summary replaces the bulk of the history, keeping the context window lean without losing continuity. The compaction cost is tracked per tenant.

## 4. System Architecture

### 4.1 High-level diagram

```

Claude (host)
  |
  |  handle_message(tenant_id, entity_id, message)
  v
BaseEdgeMCPServer  (MCP over stdio)
  |
  v
EdgeAgent
  |-- StateMachine      (stage: COLD -> INTERESTED -> CLOSING -> WON)
  |-- Memory            (per-entity conversation + state)
  |-- TenantStore       (name, product, pricing, language, plan)
  |-- UsageTracker      (calls, tokens, cost per tenant per month)
  |
  v
Internal LLM loop
  |-- system prompt (built from tenant config + current stage)
  |-- domain tools  (analyze, update_crm, schedule_followup ...)
  |-- up to N iterations until end_turn or tool loop limit
  |
  v
Final text reply  (returned to host)

```

### 4.2 Standard MCP tools (every vertical gets these for free)

| Tool             | Description                                                  |
|------------------|--------------------------------------------------------------|
| handle_message   | Main entry: runs the full internal agent loop, returns reply |
| get_entity_state | Current state machine stage + all metadata for an entity     |
| get_conversation | Message history (paginated by last N)                        |
| set_entity_stage | Manually advance or reset the pipeline stage                 |
| add_note         | Attach an internal note without triggering the loop          |
| get_usage        | Monthly calls, tokens, USD cost for a tenant                 |
| list_entities    | All entities with at least one interaction in memory         |

### 4.3 Extension points

- `extra_tools()`: add domain-specific MCP tools to any server
- `on_before_run()` / `on_after_run()`: lifecycle hooks on the agent
- `initial_entity_state()`: override the default state for new entities
- `get_context()`: dynamic per-stage prompt enrichment in the state machine
- `BaseWorker`: background process for scheduled autonomous actions (follow-ups, reminders)

## 5. Limits Overcome

| Limit in current MCP ecosystem                   | How this pattern addresses it                                         |
|--------------------------------------------------|-----------------------------------------------------------------------|
| No state between calls                           | StateMachine persists stage; Memory persists conversation + metadata  |
| Host context bloat from multi-step workflows     | Internal loop is opaque to host; only final reply surfaces            |
| Host must understand every domain to orchestrate | Host delegates ownership; specialist agent reasons on its own         |
| No multi-tenancy in MCP servers                  | TenantStore and per-tenant system prompt built in at framework level  |
| No usage metering                                | UsageTracker records every LLM call by model, tenant, and month       |
| Memory lost on restart                           | JSON (dev) or Redis (prod) backends; all state is durable             |
| Agent loops unbounded / unsafe                   | MAX_LOOP_ITERS cap; terminal stage detection; compaction at threshold |
| Hard to build new verticals                      | Four abstract classes; a new vertical is ~150 lines of domain code    |

## 6. Available Verticals

### sales-agent-mcp

A complete autonomous sales agent implementing the Edge LLM Pattern. Manages leads through a configurable sales pipeline, maintains per-lead CRM state, and generates contextually appropriate replies at each pipeline stage.

```
Pipeline:  COLD  -> INTERESTED  -> OBJECTION  -> CLOSING  -> WON
            |           |           |           |           \
            +-----+-----+-----+-----+-----> LOST
```

Internal tools (not exposed to host): analyze\_lead, generate\_reply, update\_crm, schedule\_followup.

Fully configurable per tenant: agent name, company, product description, pricing, language, usage plan.

### Planned verticals

| Package           | Domain             | Pipeline                                              |
|-------------------|--------------------|-------------------------------------------------------|
| finance-agent-mcp | Invoice lifecycle  | DRAFT -> SENT -> OVERDUE -> PAID / VOID               |
| support-agent-mcp | Ticket triage      | OPEN -> IN_PROGRESS -> WAITING -> RESOLVED            |
| hr-agent-mcp      | Candidate pipeline | APPLIED -> SCREENING -> INTERVIEW -> OFFER / REJECTED |

## 7. Production Deployment

### 7.1 Backend swap

All three persistence layers are swapped via environment variables -no code changes required:

| Layer   | Dev (default)                    | Production              |
|---------|----------------------------------|-------------------------|
| Memory  | LocalMemoryStore (JSON on disk)  | RedisMemoryStore        |
| Tenants | LocalTenantStore (JSON on disk)  | Subclass for PostgreSQL |
| Usage   | LocalUsageTracker (JSON on disk) | Subclass for PostgreSQL |

### 7.2 Claude Desktop (local)

```
{
  "mcpServers": {
    "sales": {
      "command": "uvx",
      "args": ["sales-agent-mcp"],
      "env": {
        "ANTHROPIC_API_KEY": "sk-ant-...",
        "SALES_TENANT_ID": "my-company"
      }
    }
  }
}
```

### 7.3 Server (Redis)

```
pip install 'sales-agent-mcp[redis]'
export REDIS_URL=redis://localhost:6379
export ANTHROPIC_API_KEY=sk-ant-...
sales-agent-mcp
```

## 8. Building a New Vertical

Four classes to implement. Estimated time for a simple vertical: 2-4 hours.

| Class                 | What to implement                                           | Lines of code (typical) |
|-----------------------|-------------------------------------------------------------|-------------------------|
| StateMachine          | stages, terminal_stages, transitions, _context_map          | 30-50                   |
| EdgeAgent             | build_system_prompt, get_tools, get_tool_map                | 40-80                   |
| BaseEdgeMCPServer     | create_agent, create_memory, create_tenants, create_tracker | 20-30                   |
| BaseWorker (optional) | get_due_entities, process_entity                            | 20-40                   |

All infrastructure (LLM loop, compaction, memory, multi-tenancy, usage metering, 7 standard MCP tools) is inherited from edge-llm-core at zero additional cost.

## 9. Disclaimer & Limitations

edge-llm-core and all packages built on it are ALPHA software (v0.1.0). The following limitations and risks must be understood before use in any context:

- API COSTS: Every agent loop iteration calls the Anthropic API. In production deployments with many tenants or high message volume, costs can accumulate rapidly. Always set budget alerts and monitor usage via `get_usage`.
- NO SECURITY AUDIT: This codebase has not been reviewed by a security professional. Do not use `LocalMemoryStore` or `LocalTenantStore` to persist sensitive personal data (PII, financial data, health data) in production without your own security review.
- API STABILITY: All public APIs (class signatures, method names, JSON formats) may change between minor versions in the 0.x series. Pin your dependency to a specific version in production.
- LLM BEHAVIOUR: The internal agent loop uses a generative LLM. Outputs are non-deterministic. The agent may produce incorrect, misleading, or inappropriate responses. Always implement human review for high-stakes decisions.
- LOOP SAFETY: The `MAX_LOOP_ITERS` cap (default: 10) prevents infinite loops but does not guarantee correct behaviour. An agent hitting the limit returns a sentinel string, not an error -handle this case in your integration.
- NO GDPR / COMPLIANCE GUARANTEES: Conversation history and entity state are persisted indefinitely by default. Implement your own data retention, deletion, and consent mechanisms if required by applicable law.
- ANTHROPIC TERMS: Use of this framework requires compliance with the Anthropic usage policies for the models you invoke.

---

*This document describes the architecture and design intent of edge-llm-core v0.1.0. It is provided for informational purposes only. The authors make no warranties, express or implied, regarding fitness for any particular purpose.*